Journal of Combinatorial Optimization (2025) 50:35
https://doi.org/10.1007/s10878-025-01364-6

Reinforcement learning-guided adaptive large neighborhood search for vehicle routing problem with time windows

Zhaohui Wang[1] · Qiao Cui[1] · Bin Tan[1] · Xiao Yang[2] · Weibang Zhou[2] · Xiangsheng Huang[3]

Received: 25 April 2025 / Accepted: 29 September 2025
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2025

Abstract

This paper presents PPO-ALNS, a novel hybrid framework that integrates Proximal Policy Optimization (PPO) with Adaptive Large Neighborhood Search (ALNS) for solving the Vehicle Routing Problem with Time Windows (VRPTW). Traditional heuristic methods struggle with balancing exploration and exploitation, while pure reinforcement learning approaches face challenges with discrete decision spaces and complex constraints. Our approach addresses these limitations by formulating ALNS as a Markov Decision Process, where a PPO agent dynamically guides destroy-repair operator selection, solution acceptance criteria, and search termination decisions. We design a comprehensive state representation incorporating problem-specific features and search dynamics, coupled with a four-dimensional action space for effective search coordination. The compound reward mechanism combines immediate rewards with terminal episode rewards, introducing dynamic penalties and adaptive early-stopping to optimize solution quality and computational efficiency. Experiments on VRPTW instances with 20, 50, and 100 customers demonstrate that PPO-ALNS consistently outperforms traditional ALNS, Ant Colony Optimization (ACO), and other Reinforcement Learning (RL)-based operator selection methods, achieving improvements of 11.37–17.41% over baseline approaches. The performance advantage increases with problem complexity, indicating strong scalability. Ablation studies validate the synergistic effect of simultaneous operator learning, while sensitivity analysis confirms optimal performance in medium time window scenarios, highlighting the framework's effectiveness for complex routing optimization.

Keywords Reinforcement learning · Adaptive large neighborhood search · Vehicle routing problem with time windows · Proximal policy optimization

Extended author information available on the last page of the article

Published online: 25 October 2025

1 Introduction

Vehicle Routing Problem (VRP) is one of the important combinatorial optimization problems where an optimal route to visit customers is constructed. This problem plays a significant role in various logistics and transportation systems, making it a hot topic in both operations research and computer science (Kim et al. 2015). The practical applications of VRP are expanding rapidly, including parcel delivery (Enthoven et al. 2020), urban waste collection (Akbarpour et al. 2021), and ride-sharing services (Martins et al. 2021).

The basic VRP model assumes that vehicles can serve each demand point at any time without specific time requirements. However, in many real-world problems, there are often time window constraints (Time Windows, TW), meaning that customers require service to be completed within a specific time period. This additional time constraint significantly increases the complexity of the problem, forming the Vehicle Routing Problem with Time Windows (VRPTW), which was proposed by Solomon (1987). In VRPTW, each vehicle not only needs to plan its route reasonably but also must ensure that it arrives at the customer within the specified time window, avoiding penalties or additional costs associated with early or late service. Therefore, VRPTW is considered a more complex and challenging combinatorial optimization problem than the classic VRP (Kosolsombat and Ratanavilisagul 2023).

In recent years, various approaches have been proposed by academia and industry to solve the VRPTW, including exact algorithms (Toth and Vigo 2002; Archetti et al. 2011; Costa et al. 2019; Baldacci et al. 2012), heuristic algorithms (Solomon 1987; Mei et al. 2017), and metaheuristic algorithms (Ursani et al. 2011; Zhong and Pan 2007; Ho and Haugland 2004; Dixit et al. 2019; Gambardella et al. 1999). Although exact algorithms can theoretically find the optimal solution, their computational time increases exponentially with the problem size, making them impractical for large instances. Heuristic and metaheuristic algorithms strike a balance between computational efficiency and solution quality, but they still face limitations when dealing with complex VRPTW, especially in terms of balancing solution quality and computational time for large-scale problems with multi-dimensional constraints (Li et al. 2021). Recently, optimization methods based on deep learning and reinforcement learning (RL) have emerged, showing promising application prospects (Kosolsombat and Ratanavilisagul 2023; Falkner and Schmidt-Thieme 2020; Deng et al. 2024; Wang et al. 2023; Wu et al. 2024). However, these methods still face challenges in handling discrete decision spaces and complex time window constraints. Additionally, traditional RL algorithms often lack global search capabilities and are prone to getting stuck in local optima.

To overcome these limitations, hybrid algorithms combining RL and heuristic methods have been explored (Fernandes et al. 2009; Zhao et al. 2021; Gao et al. 2020). These hybrid methods can be roughly divided into two types, one is to use heuristics to improve the solutions generated by end-to-end RL methods, and the other is to use RL methods as a support tool to solve the internal subproblems of heuristics (Bogyrbayeva et al. 2024). These hybrid approaches aim to leverage the strengths of both methods: the global search capability of RL and the local refinement ability of heuristic algorithms. However, existing hybrid algorithms still face issues

such as insufficient exploration of the solution space, a tendency to get trapped in local optima, and slow performance when solving large-scale problems. Additionally, the integration of these two methods can be challenging, requiring careful design to ensure that the strengths of each approach are effectively utilized.

Generally, the main contributions of this research are summarized as follows:

- We propose a novel hybrid framework (PPO-ALNS) that integrates Proximal Policy Optimization with Adaptive Large Neighborhood Search. Our framework enables the PPO agent to dynamically guide the selection of destroy-repair operators, solution acceptance criteria, and algorithm termination timing based on real-time problem states, achieving synergistic collaboration between policy learning and heuristic search.
- We design a comprehensive state-action space specifically tailored for VRPTW, incorporating multidimensional features such as time window constraint tightness, customer spatial distribution patterns, historical operator performance statistics, and search trajectory characteristics. Our four-dimensional action space facilitates intelligent coordination of exploration-exploitation trade-offs during ALNS iterations, enhancing adaptive decision-making capabilities.
- We develop an innovative compound reward mechanism that combines immediate step-wise rewards with terminal episode rewards. Our design introduces dynamic penalties for rejecting superior solutions and accepting inferior ones, along with an adaptive early-stopping reward mechanism that optimizes the balance between solution quality and computational efficiency. This approach effectively prevents premature convergence while preserving search diversity throughout the optimization process.

In addition to addressing complex logistics challenges, the proposed PPO-ALNS framework is also highly relevant for cross-domain emergency planning and scheduling. In scenarios such as natural disasters, public health emergencies, or other urgent events, timely and efficient decision-making is critical. The ability of our method to rapidly re-optimize routes and resource allocation in response to dynamically changing conditions underscores its potential as a valuable tool for emergency management. By facilitating real-time data integration and adaptive decision-making, our approach can contribute to reducing response times and enhancing coordination in high-stakes operational environments.

The remainder of this paper is organized as follows: Sect. 2 introduces the related work of VRPTW and ALNS. Section 3 describes the principles of PPO and ALNS, along with the design process of the hybrid algorithm. Section 4 presents the experimental setup and results analysis. Finally, Section 5 concludes the paper and discusses future research directions. Code is available at https://github.com/Kikujjy/ppo-alns.

2 Related work

In this section, we first review the existing methods for solving VRPTW problems, including precision algorithms, heuristics, RL methods, and hybrid methods, and point out their limitations. Then, we discuss the related research in the field of ALNS, especially the method of operator selection. Finally, we explain how to combine RL methods with ALNS.

2.1 Algorithms for VRPTW

VRPTW is an extension of the classical VRP, where customers must be served within specific time windows. Due to its significance in real-world applications such as transportation, delivery services, and supply chain management, VRPTW has become a key research topic in logistics and operations research (Liu et al. 2023). Compared to VRP, VRPTW incorporates time window constraints, requiring each vehicle to arrive at the customer location within a specified time frame, which complicates the optimization process.

Exact algorithms including Branch-and-Bound (Toth and Vigo 2002) and Branch-and-Price (Costa et al. 2019) guarantee optimal solutions for small instances but become computationally intractable for larger problems due to exponential search space growth. Consequently, heuristic and metaheuristic approaches dominate practical applications.

Early heuristics such as Solomon (1987)'s insertion method and Clarke & Wright savings algorithm (Republic et al. 2016) provide computational efficiency but often yield suboptimal solutions due to local decision rules. Metaheuristic algorithms including Genetic Algorithms (GA) (Ursani et al. 2011), Simulated Annealing (SA) (Zhong and Pan 2007), Ant Colony Optimization (ACO) (Dorigo et al. 2006; Gambardella et al. 1999), and Tabu Search (TS) (Ho and Haugland 2004) achieve better global optimization. Large Neighborhood Search (LNS) (Shaw 1998) and its adaptive variant Adaptive Large Neighborhood Search (ALNS) (Pisinger and Ropke 2007) have emerged as leading metaheuristics, dynamically adjusting search strategies for enhanced flexibility.

However, these methods require extensive parameter tuning and struggle with exploration-exploitation balance (Zhang et al. 2022). Recent machine learning advances have introduced RL techniques to routing problems. Nazari et al. (2018) demonstrated Deep Reinforcement Learning (DRL)'s competitiveness with traditional heuristics, while Kool et al. (2019) extended this using attention models and REINFORCE for large-scale instances. Wu et al. (2022) developed self-attention-based improvement heuristics for the Traveling Salesman Problem (TSP) and Capacitated Vehicle Routing Problem (CVRP), significantly outperforming existing methods.

RL applications to VRPTW include Falkner and Schmidt-Thieme (2020)'s Joint Attention Model for Parallel Route-Construction (JAMPR), which constructs multiple routes concurrently. While RL offers adaptive optimization without problem-specific rules, challenges remain in convergence speed, sample complexity, and discrete action space handling. PPO (Schulman et al. 2017) addresses some limi-

tations through constrained policy updates and improved stability, but alone may struggle with VRPTW's vast solution space and hard constraints.

Hybrid algorithms combining RL with metaheuristics (Fernandes et al. 2009; Zhao et al. 2021; Gao et al. 2020) leverage complementary strengths: RL learns efficient strategies through dynamic decision-making, while metaheuristics explore solution spaces on a large scale.

Cappart et al. (2021) integrated RL with constraint programming to solve VRPTW using dynamic programming approaches. Zhao et al. (2021) combined AC algorithms with local search for large-scale VRP optimization. Chen et al. (2020) developed Variable Neighborhood Search with RL for periodic VRPTW. Qin et al. (2021) proposed a RL-based hyper-heuristic algorithm for heterogeneous VRP, demonstrating excellent performance on large-scale problems. Gupta et al. (2022) introduced DRL with attention-based encoder-decoder frameworks for capacitated vehicle routing problem with time windows (CVRPTW), balancing solution quality and computational efficiency.

Hottung and Tierney (2020) proposed Neural Large Neighborhood Search combining deep learning with RL for CVRP and split delivery vehicle routing problem (SDVRP). Lopes Silva et al. (2019) developed multi-agent framework AMAM for VRPTW using RL and meta-heuristic cooperation. Da Costa et al. (2021) applied DRL to learn 2-opt strategies for TSP and VRP variants.

Despite progress, existing hybrid methods exhibit limited solution space exploration, local optima entrapment, and computational inefficiency for large-scale problems.

2.2 Adaptive large neighborhood search

Since its introduction by Ropke and Pisinger (2006), ALNS has become a core metaheuristic framework for solving VRP and its variants. Its core mechanism dynamically adjusts the selection probabilities of destroy and repair operators to balance exploration and exploitation during the search process.

Classical ALNS employs history-based operator selection, categorizing performance into three groups: global optimum improvements (σ_1), better solution acceptance (σ_2), and mediocre solution acceptance (σ_3), with distinct reward values and roulette wheel selection (Ropke and Pisinger 2006). Meta-analysis by Turkeš et al. (2021) shows adaptive layers achieve marginal 0.14% improvement, recommended only for specialized contexts due to limited effectiveness and increased complexity.

Destroy and repair operator design critically governs ALNS adaptability and optimization efficacy. Common destroy strategies include randomness-driven, cost-based elimination, and sequence-oriented destruction, while repair operations employ greedy heuristics, regret-value prioritization, or stochastic logic. Empirical analysis reveals sequence-based destroy operators and regret-enhanced repair strategies demonstrate superior performance (Voigt 2025). Hybrid configurations combining conventional destroy sets with sequence-aware mechanisms achieve enhanced outcomes while maintaining solution diversity.

Solution acceptance criteria critically impact ALNS performance. Dynamic criteria permitting conditional inferior solution acceptance–simulated annealing (SA),

Record-to-Record Travel (RRT), and Threshold Acceptance (TA)–generally outperform greedy strategies (Windras Mara et al. 2022). SA remains prevalent due to local optima escape effectiveness. Santini et al. (2018) demonstrate linear RRT achieves robust performance across vehicle routing, minimum spanning tree, and quadratic assignment problems, while random walk and hill climbing underperform due to insufficient exploration-exploitation balance. Dynamic criteria integration enhances ALNS-based heuristics' robustness and solution quality.

Termination criteria selection balances computational efficiency and solution quality, considering resource constraints and performance objectives (Windras Mara et al. 2022). Common strategies include fixed iteration limits, dynamic stagnation thresholds, runtime restrictions, and temperature thresholds. Fixed iterations offer simplicity, while dynamic termination enables adaptive control based on solution stagnation patterns. Termination criteria must be aligned with specific problem characteristics and practical resource limitations to achieve optimal trade-offs between computational efficiency and solution quality.

2.3 Combine RL with ALNS

Recent advancements in integrating RL with ALNS have marked a pivotal breakthrough in combinatorial optimization, where RL's dynamic decision-making capabilities enhance ALNS's heuristic search process to improve adaptability and solution efficiency. Reijnen et al. (2024) proposed the DR-ALNS framework, pioneering DRL for online dynamic configuration of ALNS, including operator selection, destruction intensity adjustment, and SA criteria control. Its design demonstrates superior performance over traditional ALNS and Bayesian optimization methods with strong generalization capabilities. Kallestad et al. (2023) developed the DRLH framework, replacing ALNS's conventional roulette-wheel mechanism with DRL-based decision-making. Their approach robustly handles large operator sets across multiple combinatorial optimization problems while outperforming standard ALNS. Johnn et al. (2024) introduced Graph Neural Networks (GNNs) for state representation in their GRLOS method, managing complex combinations of 28 destroy/7 repair operators across five routing problem types. The graph-structured RL policy maintains efficiency even in 1,000-node instances. Wang et al. (2025) designed hybrid state features for the Capacitated Electric Vehicle Routing Problem (CEVRP), integrating charging strategy selection into DRL-driven ALNS, achieving 7 out of 8 optimal solutions in medium-scale instances. These studies advance the integration of RL and ALNS by overcoming limitations of traditional weight-based mechanisms, with notable progress in state representation, operator generalization, and complex constraint handling.

3 Methodology

Our proposed PPO-ALNS framework establishes intelligent control over combinatorial optimization processes through DRL. The framework formulates the ALNS algorithm as a Markov Decision Process (MDP) environment, where a PPO-driven

agent executes four-dimensional decisions at each iteration: (1) Dynamically selects destroy operators from a predefined library based on current solution characteristics; (2) Adaptively matches repair operator combinations according to post-destruction solution structures; (3) Evaluates solution quality and search status to determine acceptance of new solutions as subsequent search baselines; (4) Monitors convergence patterns to dynamically decide early termination for balancing exploration and computational efficiency. This learning-based approach aims to improve key components of the ALNS algorithm by decoupling operator selection and solution acceptance decisions from traditional heuristic rules, thereby reducing limitations imposed by handcrafted rules. The overall framework is illustrated in Algorithm 1, which demonstrates how the PPO agent learns efficient policies that are then integrated into the ALNS process for solving VRPTW problems.

Input: VRPTW instance, policy network π_θ, destroy operators Ω^-,
repair operators Ω^+, maximum iterations T, early stopping threshold k
Initialize: $x_{best} \leftarrow$ initial solution x,
$x_0 \leftarrow x_{best}$
$termination_counter \leftarrow 0$
Extract initial state s_0 from $x_{current}$
for $t = 1$ to T **do**
    Select actions $(d_t, r_t, a_t, e_t) \sim \pi_\theta(\cdot|s_t)$
    if $a_t = 1$ **then**
        $x \leftarrow x_{t-1}$
    end if
    Apply destroy operator $d_t \in \Omega^-$ to x to obtain partial solution
    Apply repair operator $r_t \in \Omega^+$ to partial solution to obtain x_t
    if $cost(x_t) < cost(x_{best})$ **then**
        $x_{best} \leftarrow x_t$
    end if
    Extract next state s_{t+1} from x_t
    Calculate reward R_t according to reward function
    if $e_t = 1$ **then**
        $termination_counter \leftarrow termination_counter + 1$
    else
        $termination_counter \leftarrow 0$
    end if
    if $termination_counter \geq k$ **then**
        break
    end if
    $s_t \leftarrow s_{t+1}$
end for
Return x_{best}

Algorithm 1 PPO-ALNS Framework for VRPTW

3.1 ALNS

The ALNS framework operates through iterative cycles of destruction and reconstruction of solutions. Each iteration applies a paired destroy operator $o_i^- \in D$ and repair operator $o_i^+ \in R$, which collaboratively generate a perturbed solution from the current incumbent. A critical parameter, the destroy scale d, governs the number of elements (e.g., nodes in routing problems) removed during destruction and subsequently reinserted during reconstruction. This parameter, either fixed as a hyper-

parameter or dynamically sampled, directly influences the magnitude of solution perturbations.

The algorithm partitions the search process into sequential segments of fixed length (i.e., a predefined number of iterations). At the start of each segment K, all operators are initialized with a score $\psi_i = 0$. During the segment, scores for operators involved in generating new incumbent solutions are incremented based on solution quality: δ_1 for a new global best, δ_2 for a local improvement ($\delta_1 > \delta_2$), or δ_3 for accepted inferior solutions ($\delta_2 > \delta_3$). Let N_i denote the selection count of operator o_i within segment K. The weight $w_{i,K}$ for the next segment $K+1$ is updated via an exponential moving average to balance historical performance and recent effectiveness:

$$w_{i,K+1} = \begin{cases} (1-\alpha_{RW})w_{i,K} + \alpha_{RW}\frac{\psi_i}{N_i} & \text{if } \psi_i > 0, \\ w_{i,K} & \text{otherwise,} \end{cases} \tag{1}$$

where $\alpha_{RW} \in [0,1]$ is a reaction factor controlling the adaptation rate. Operator selection probabilities within each segment are proportional to their normalized weights: $P(o_i^-) = \frac{w_{i,K}^-}{\sum_{j\in D} w_{j,K}^-}$ for destroy operators and analogously for repair operators. This adaptive mechanism prioritizes high-performing operators while preserving stochastic exploration.

New solutions are evaluated using a SA-inspired acceptance criterion to escape local optima. For a minimization problem, a candidate solution s' with objective value $F(s')$ is accepted over the current solution s with probability $\min\left(1, \exp\left(\frac{F(s)-F(s')}{\tau}\right)\right)$, where τ is a temperature parameter initialized to τ_0. The temperature decays geometrically as $\tau \leftarrow c_t\tau$ per iteration ($c_t < 1$), gradually favoring exploitation over exploration.

The algorithm terminates upon meeting predefined stopping criteria, such as maximum iterations, computational time limits, or convergence thresholds. This framework balances adaptability (through dynamic operator weighting) and diversification (via SA acceptance), enabling effective navigation of complex solution spaces in combinatorial optimization problems.

3.2 Markov decision process formulation

We formalize the ALNS operator selection process as an MDP $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R} \rangle$, where states encode the search progress and actions determine operator sequences. The state space $\mathcal{S}$ captures essential search dynamics through the current solution quality, historical operator performance, and problem-specific features. The action space $\mathcal{A}$ comprises operator selection decisions, including destroy-repair pairs and acceptance criteria adjustments.

The reward function $\mathcal{R}(s_t, a_t)$ quantifies immediate search improvement using a weighted combination of objective value gain Δf and computational effort Δt:

$$r_t = \eta \cdot \Delta f + (1-\eta) \cdot (-\Delta t), \quad \eta \in [0,1] \tag{2}$$

State transitions $\mathcal{P}(s_{t+1}|s_t, a_t)$ reflect the stochastic nature of solution modifications by operators. The agent learns a policy π_θ to maximize the expected discounted return $\mathbb{E}[\sum_{k=0}^{\infty} \gamma^k r_{t+k}]$ with discount factor $\gamma \in [0, 1)$. This formulation strategically balances immediate rewards with long-term search effectiveness through operator sequencing.

State space S provides the DRL agent with the necessary information to make informed decisions for selecting the optimal actions during each search iteration. To achieve this, we define S as a vector containing several features, as detailed in Appendix A. These features are designed to give the agent pertinent insights into the search process. Specifically, they include information such as the current progress of the search, the improvement coefficient compared to the previous iteration, and the initial and best-found objective function values. Furthermore, the usage frequencies of both destroy and repair operators are provided, allowing the agent to understand which operators have been applied most frequently. Additionally, the state space incorporates relevant problem-specific features, such as demand, time windows, service times, and travel times for the VRPTW. These features allow the agent to have a complete overview of the environment, aiding in making better decisions for the next iteration.

Action space A is composed of four distinct action spaces: A_1 is the **destroy operator selection**, where the agent selects one destroy operator from the set of operators configured for the problem at hand, to apply to the current solution. A_2 is the **repair operator selection**, where the agent selects a repair operator from the set of available operators, which is applied to a destroyed solution. A_3 is the **acceptance criterion**, which determines whether the new solution generated by the current search procedure should be accepted. The agent evaluates if the newly generated solution should replace the current best-found solution. A_4 is the **search termination signal**, where the agent decides whether to terminate the search process or continue with further iterations.

Reward function R is designed to guide the agent in selecting actions based on the current state S of the search process. The reward at each iteration is given by:

$$R_t = \begin{cases} -\alpha \cdot \frac{c(x_t)-c(x_{t-1})}{c(x_{t-1})}, & \text{if } A_{3t} = 0 \text{ and } c(x_t) < c(x_{t-1}), \\ \beta \cdot \frac{c(x_t)-c(x_{t-1})}{c(x_{t-1})}, & \text{if } A_{3t} = 1 \text{ and } c(x_t) > c(x_{t-1}), \\ \gamma \cdot \frac{c(x_t)-c(x_{\text{best}})}{c(x_{\text{best}})}, & \text{if } c(x_t) < c(x_{\text{best}}), \end{cases} \tag{3}$$

where $c(x_t)$ represents the objective function value at iteration t, $c(x_{t-1})$ is the objective value of the previous iteration, and $c(x_{\text{best}})$ denotes the best-found solution. The parameters α, β, and γ are positive constants that control the magnitude of the reward for different conditions.

At the end of the search, a final reward R_{final} is calculated as follows:

$$R_{\text{final}} = f_1 \cdot \frac{c(x_{\text{best}}) - c(x_{\text{init}})}{c(x_{\text{init}})} + f_2 \cdot \frac{t}{T_{\max}}, \tag{4}$$

where $c(x_{\text{init}})$ is the initial objective value, $c(x_{\text{best}})$ is the best-found solution, t is the number of iterations taken, and $T_{\max}$ is the maximum number of allowed iterations. The parameters f_1 and f_2 are scaling factors that control the relative influence of the improvement in the objective value and the number of iterations taken.

3.3 Proximal policy optimization

PPO algorithm (Schulman et al. 2017) was employed to train a policy that adaptively controls the ALNS framework for solving the VRPTW. PPO is a RL algorithm that aims to improve the stability and reliability of policy gradient methods. PPO achieves this by using a surrogate objective function that restricts the policy update to be within a certain range, preventing large, destabilizing updates. This section provides a detailed description of the PPO algorithm, including its mathematical formulation and implementation details.

The PPO algorithm optimizes a policy $\pi_\theta(a|s)$, parameterized by θ, to maximize the expected cumulative reward. The objective function for PPO is defined as:

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right] \tag{5}$$

where $r_t(\theta)$ is the probability ratio between the new and old policies:

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} \tag{6}$$

and $\hat{A}_t$ is the advantage estimate at time step t. The clipping function $\text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)$ ensures that the probability ratio $r_t(\theta)$ stays within the interval $[1-\epsilon, 1+\epsilon]$, where ϵ is a hyperparameter that controls the range of the policy update.

The advantage estimate $\hat{A}_t$ is computed using the Generalized Advantage Estimation (GAE) method:

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l} \tag{7}$$

where γ is the discount factor, λ is the GAE parameter, and δ_t is the temporal difference (TD) error:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t) \tag{8}$$

Here, $V(s_t)$ is the value function, which is also parameterized and learned alongside the policy.

The PPO algorithm alternates between sampling data through interaction with the environment and optimizing the policy using the collected data.

In summary, the PPO algorithm improves the stability and performance of policy gradient methods by using a clipped surrogate objective function. This prevents large updates that could destabilize the learning process. By alternating between data collection and policy optimization, PPO effectively learns policies that maximize the expected cumulative reward in a stable and efficient manner.

4 Experiment

4.1 Experiment details

4.1.1 Data generation

We generate VRPTW instances of sizes 20, 50, and 100 for model training and testing. Customer node coordinates are randomly generated from a uniform distribution over $[0, 1] \times [0, 1]$. The depot coordinates are determined as the geometric center of all customer nodes. We assume sufficient vehicle availability (set equal to the number of customer nodes) with a fixed vehicle capacity of 64. Customer demands are randomly generated from the interval [1, 16], service times from [0.05, 0.1], and time windows from [0.1, 0.3]. The maximum travel time is set to 1. All pairwise Euclidean distances between customer nodes are normalized through division by a scaling factor of 50.

4.1.2 Operator sselection

Our PPO-ALNS framework integrates five destroy operators and three repair operators to dynamically adjust the search process.

Destroy Operators: Random Destroy, String Destroy, Route Destroy, Worst-Cost Destroy, and Sequence Destroy.

Repair Operators: Greedy Repair, Criticality-Based Repair, and Regret-k Repair.

Detailed descriptions of these operators are provided in Appendix B. The PPO agent learns to adaptively select operators based on real-time solution states, balancing exploration and exploitation during the search.

4.1.3 Initial solution

Two independent initialization strategies are employed to generate initial solutions for comparative experiments:

- **Greedy Initialization**: A feasible solution is constructed by iteratively inserting the nearest customer node that satisfies both time window and capacity constraints. This method prioritizes rapid solution generation but may converge to local optima.
- **Clarke-Wright (CW) Savings Initialization**: The solution is generated through route consolidation based on the CW savings algorithm (Clarke and Wright 1964), which maximizes the *savings value* – the travel distance reduction achieved by

merging two routes. This method emphasizes global structural optimization at the cost of higher computational complexity.

4.1.4 Baseline algorithms

We compare our proposed PPO-ALNS with the following baseline algorithms:

Adaptive Large Neighborhood Search ALNS is a metaheuristic algorithm that iteratively improves solutions by dynamically selecting and applying destroy and repair operators. In the context of the VRPTW, ALNS removes a subset of customers from the current solution using adaptive destruction heuristics and reinserts them using repair operators. The selection of operators is guided by a roulette wheel mechanism that prioritizes historically high-performing operators, enabling the algorithm to balance exploration and exploitation. ALNS excels in handling complex VRPTW constraints, such as time windows and vehicle capacity, by leveraging its adaptive nature to escape local optima and systematically refine solutions.

Ant Colony Optimization ACO is a swarm intelligence-based metaheuristic inspired by the foraging behavior of ants. For VRPTW, artificial ants construct feasible routes by probabilistically selecting the next customer based on pheromone trails and heuristic information (e.g., proximity or time window urgency). Pheromone levels on edges are updated to reflect solution quality, reinforcing paths associated with low-cost solutions. The algorithm inherently balances exploration (via probabilistic selection) and exploitation (via pheromone-guided intensification), making it effective for combinatorial optimization problems like VRPTW. ACO's ability to handle dynamic constraints and generate diverse solutions through parallel agent-based search has established it as a robust baseline for route optimization in time-constrained environments.

4.1.5 Parameter settings

For all experiments, we employ 32 parallel environments for asynchronous training with a total of 1,600,000 training steps. The initial learning rate is set to 0.001 using the Adam optimizer, with a batch size of 128. The discount factor γ is configured as 0.99, and the maximum number of iterations is limited to 100.

The PPO algorithm utilizes a Multi-Layer Perceptron (MLP) architecture for both policy and value networks, which share identical feature extractors. These networks consist of three fully-connected layers with [512, 256, 128] hidden units and ReLU activation functions, excluding the final output layers. The policy network's final linear layer projects to the action space dimension, while the value network's final layer produces a scalar output.

During testing, the ALNS baseline algorithm's iteration count is synchronized with PPO-ALNS. The adaptive mechanism employs roulette wheel scores [25, 5, 1, 0] corresponding to four quality levels of accepted solutions, accompanied by a decay coefficient of 0.9 for score adjustment.

4.2 Experiment results

This section presents a comprehensive analysis of the experimental results. All experiments were conducted on a machine equipped with an RTX 3090 GPU and an Intel(R) Xeon(R) CPU E5-2678 v3 @ 2.50GHz processor. The average training times for our approach were approximately 1 h for 20-node instances, 1.5 h for 50-node instances, and 3.5 h for 100-node instances. We evaluate our proposed method against various baseline approaches across 128 test instances divided into three problem sizes (20, 50, and 100 customer nodes). Table 1 summarizes the overall performance comparison, including average objective values (Aver.), improvement percentages relative to ALNS with Greedy initialization (Impr.), and computational time in seconds (Time).

4.2.1 Solution quality analysis

The experimental results demonstrate that our proposed method with CW initialization consistently achieves the best solution quality across all problem sizes, as evidenced by the lowest average objective values (6.01, 11.92, and 20.45 for $n = 20$, $n = 50$, and $n = 100$, respectively). This represents significant improvements of 11.37%, 13.15%, and 17.41% over the baseline ALNS with Greedy initialization, which serves as our reference point.

The substantial improvement in solution quality can be attributed to the RL agent's ability to adaptively select appropriate operator pairs based on the current search state, thereby more effectively balancing exploration and exploitation throughout the search process. This adaptive selection mechanism proves particularly beneficial as problem size increases, with the improvement margin expanding from 11.37% for $n = 20$ to 17.41% for $n = 100$. This scalability advantage highlights the effectiveness of our learning-based approach in handling larger and more complex VRPTW instances. Notably, our method outperforms not only traditional ALNS approaches

Table 1 Performance comparison for VRPTW across different methods

Method	$n = 20$			$n = 50$		
	Aver	Impr	Time	Aver	Impr	Time
ALNS(Greedy)	6.78	–	2.85	13.73	–	11.30
ALNS(CW)	6.33	6.57%	2.81	12.48	9.11%	10.04
ACO	6.39	5.68%	52.85	13.48	1.79%	207.00
GRLOS(CW)	6.11	9.89%	46.09	12.15	11.51%	88.47
Ours(Greedy)	6.14	9.37%	12.02	12.53	8.71%	41.07
Ours(CW)	**6.01**	11.37%	14.54	**11.92**	13.15%	38.95

Method	$n = 100$		
	Aver	Impr	Time
ALNS(Greedy)	24.76	–	66.29
ALNS(CW)	21.84	11.81%	52.11
ACO	25.30	−2.18%	736.00
GRLOS(CW)	21.65	12.58%	212.38
Ours(Greedy)	22.00	11.17%	166.16
Ours(CW)	**20.45**	17.41%	193.48

but also the recent GRLOS(CW) method, which proposed by Johnn et al. (2024) and achieves improvements of 9.89%, 11.51%, and 12.58%, respectively. Furthermore, GRLOS employs a soft constraint approach with time window violation penalties, which may result in solutions that do not fully satisfy time window constraints. Our algorithm circumvents this limitation by maintaining strict constraint adherence throughout the optimization process.

When examining the impact of initialization methods, we observe that CW initialization consistently outperforms Greedy initialization for both ALNS and our proposed approaches. For instance, ALNS with CW initialization achieves improvement percentages of 6.57%, 9.11%, and 11.81% across the three problem sizes, while our method with CW initialization further enhances these improvements to 11.37%, 13.15%, and 17.41%, respectively. This indicates that a higher-quality initial solution provides a more favorable starting point for subsequent search processes, enabling both methods to reach better final solutions.

4.2.2 Computational efficiency analysis

Regarding computational efficiency, our results reveal a clear trade-off between solution quality and computational time across the different methods. The baseline ALNS approaches (with both Greedy and CW initialization) exhibit the lowest computational times (2.81-2.85s for $n = 20$, 10.04-11.30s for $n = 50$, and 52.11-66.29s for $n = 100$). However, these methods also yield inferior solution quality compared to our proposed approaches.

ACO method demonstrates significantly higher computational requirements (52.85s, 207.00s, and 736.00s for the three problem sizes), while achieving modest solution quality improvements. This highlights ACO's computational intensity, which grows substantially with problem size due to its population-based search mechanism. Notably, ACO even produces worse results than the baseline for $n = 100$ instances (with a negative improvement of -2.18%), despite requiring much longer computational time.

GRLOS(CW) method requires moderate computational time (46.09s, 88.47s, and 212.38s for the three problem sizes) while achieving reasonable solution quality improvements of 9.89%, 11.51%, and 12.58% respectively. However, our proposed approaches still outperform GRLOS in terms of both solution quality and computational efficiency.

Our proposed approaches strike a favorable balance between solution quality and computational efficiency. Though requiring moderately higher computational time than baseline ALNS, our methods deliver substantially better solution quality while remaining competitive in terms of computational efficiency. For instance, our method with CW initialization requires 14.54s, 38.95s, and 193.48s for the three problem sizes while achieving superior improvements of 11.37%, 13.15%, and 17.41% respectively.

4.2.3 Pairwise performance comparison

Figure 1 presents pairwise comparison results, where each cell indicates the percentage of instances in which the row method outperforms the column method. The heat maps demonstrate the robustness and superiority of our proposed approach across different problem sizes.

Our method with CW initialization consistently outperforms all baseline approaches across all problem scales. For smaller instances ($n = 20$), it achieves strong win rates against traditional ALNS methods (75.8–93.0%) and competitive performance against other RL-based operator selection methods. The performance advantage becomes more pronounced as problem complexity increases, with win rates reaching 96.9–100.0% against traditional methods for larger instances ($n = 100$).

The results reveal three key insights: (1) our approach demonstrates excellent scalability, with performance advantages increasing alongside problem size; (2) CW initialization consistently outperforms Greedy initialization, validating the importance of high-quality initial solutions; and (3) compared to other RL-based approaches for operator selection, our method maintains competitive advantages across all problem scales, particularly excelling in large-scale instances where the win rate reaches 77.3%.

	ALNS(Greedy)	ALNS(CW)	ACO	GRLOS(CW)	Ours(Greedy)	Ours(CW)
ALNS(Greedy)	0	24.2	23.4	26.6	10.9	7.0
ALNS(CW)	75.8	0	58.6	41.4	33.6	24.2
ACO	76.6	41.4	0	33.6	25.0	15.6
GRLOS(CW)	73.4	58.6	66.4	0	52.3	46.1
Ours(Greedy)	89.1	66.4	75.0	47.7	0	36.7
Ours(CW)	93.0	75.8	84.4	53.9	63.3	0

(a) Comparison for size 20

	ALNS(Greedy)	ALNS(CW)	ACO	GRLOS(CW)	Ours(Greedy)	Ours(CW)
ALNS(Greedy)	0	13.3	47.7	12.5	15.6	1.6
ALNS(CW)	86.7	0	85.9	38.3	50.8	26.6
ACO	52.3	14.1	0	13.3	10.9	3.1
GRLOS(CW)	87.5	61.7	86.7	0	61.7	44.5
Ours(Greedy)	84.4	49.2	89.1	38.3	0	18.0
Ours(CW)	98.4	73.4	96.9	55.5	82.0	0

(b) Comparison for size 50

	ALNS(Greedy)	ALNS(CW)	ACO	GRLOS(CW)	Ours(Greedy)	Ours(CW)
ALNS(Greedy)	0	4.7	64.1	3.9	5.5	0.0
ALNS(CW)	95.3	0	100.0	42.2	57.8	3.1
ACO	35.9	0.0	0	0.8	2.3	0.0
GRLOS(CW)	96.1	57.8	99.2	0	54.7	22.7
Ours(Greedy)	94.5	42.2	97.7	45.3	0	9.4
Ours(CW)	100.0	96.9	100.0	77.3	90.6	0

(c) Comparison for size 100

Fig. 1 The proportion of all instances which the result of the method on the row outperforms the method on the column

4.3 Ablation study

To evaluate the individual contribution of each component in our framework, we conduct an ablation study where PPO guides either the destroy or repair operator selection while the other uses traditional ALNS selection mechanisms. Table 2 presents the results of this analysis.

The Ours-D(CW) configuration, which applies PPO guidance solely to destroy operators, achieves improvements of 3.55%, 3.33%, and 5.93% across the three problem sizes. The Ours-R(CW) configuration, where PPO guides only repair operators, demonstrates improvements of 3.08%, 3.05%, and 4.79%. These results indicate that both destroy and repair operator selection contribute meaningfully to performance enhancement, with destroy operator guidance showing slightly higher impact, particularly for larger instances.

Most significantly, our complete method (Ours(CW)) that integrates PPO guidance for both destroy and repair operators achieves improvements of 5.13%, 4.45%, and 6.34%, which exceed the performance gains of either individual component. This validates the synergistic effect of simultaneous operator learning, demonstrating that the coordinated optimization of both search phases enables more sophisticated search behavior than optimizing either phase independently.

4.4 Sensitivity analysis on time window tightness

To evaluate the robustness and performance of our proposed PPO-ALNS algorithm under varying temporal constraints, we conducted a sensitivity analysis on the tightness of customer time windows. We categorized the problem instances into three groups based on the width of the time windows: small (*S*-window), medium (*M*-window), and large (*L*-window). These categories correspond to time window widths randomly generated from the intervals [0.06, 0.16], [0.1, 0.3], and [0.3, 0.5], respectively. These window widths represent approximately 10%, 20%, and 40% of the maximum travel time. The comparative results against the baseline ALNS are detailed in Table 3.

The results reveal that PPO-ALNS consistently outperforms the baseline ALNS across all scenarios, with cost reductions ranging from 2.15% to 6.34%, achieving a mean improvement of 3.94% over ALNS. The superiority of PPO-ALNS is most pronounced in the M-window instances, where it achieves a mean cost reduction of 5.31% over ALNS. This suggests that our algorithm excels in scenarios with a moderate degree of flexibility. When time windows are very tight (*S*-window), the solution space is highly constrained, limiting the potential for significant optimiza-

Table 2 Ablation study: effect of PPO-guided operator selection in ALNS

Method	$n = 20$		$n = 50$		$n = 100$	
	Aver	Impr	Aver	Impr	Aver	Impr
ALNS(CW)	6.33	–	12.48	–	21.84	–
Ours-D(CW)	6.11	3.55%	12.06	3.33%	20.54	5.93%
Ours-R(CW)	6.14	3.08%	12.10	3.05%	20.79	4.79%
Ours(CW)	**6.01**	5.13%	**11.92**	4.45%	**20.45**	6.34%

Table 3 Performance comparison on time window tightness

Window	Method	$n = 20$		$n = 50$		$n = 100$	
		Aver	Impr	Aver	Impr	Aver	Impr
S-window	ALNS(CW)	7.00	–	13.69	–	23.75	–
	Ours(CW)	**6.73**	3.78%	**13.32**	2.74%	**23.24**	2.15%
M-window	ALNS(CW)	6.33	–	12.48	–	21.84	–
	Ours(CW)	**6.01**	5.13%	**11.92**	4.45%	**20.45**	6.34%
L-window	ALNS(CW)	5.65	–	10.87	–	18.83	–
	Ours(CW)	**5.42**	4.10%	**10.57**	2.78%	**18.09**	3.94%

tion for any algorithm. Conversely, when time windows are very loose (*L*-window), the problem's complexity is reduced, and the baseline ALNS can find relatively good solutions, thus narrowing the performance gap. The *M*-window setting represents a "sweet spot" of combinatorial complexity where the intelligent search capabilities of PPO-ALNS are most impactful.

4.5 Summary of findings

Our experimental evaluation demonstrates three key findings: (1) PPO-ALNS achieves consistent improvements of 11.37–17.41% over baseline methods across all problem scales, with performance advantages increasing alongside problem complexity; (2) The four-dimensional action space enables effective coordination of destroy-repair operator selection, solution acceptance, and early termination decisions, as validated by ablation studies showing synergistic effects between components; (3) The framework maintains computational competitiveness while delivering superior solution quality, particularly excelling in medium time window scenarios where intelligent operator selection provides maximum benefit. Additional experiments presented in Appendix C further validate the framework's generalization capabilities across different spatial distributions and its scalability for large-scale instances, demonstrating consistent performance improvements across diverse problem configurations.

5 Conclusion

This paper presents PPO-ALNS, a novel hybrid framework integrating Proximal Policy Optimization with Adaptive Large Neighborhood Search for VRPTW. By formulating ALNS as a MDP with a four-dimensional action space and compound reward mechanism, our approach enables intelligent coordination of destroy-repair operator selection, solution acceptance, and termination decisions.

Experimental results demonstrate PPO-ALNS's consistent superiority over traditional methods, achieving 11.37–17.41% improvements across problem scales of 20-100 customers. The performance advantage increases with problem complexity, indicating strong scalability. Ablation studies validate the synergistic effect of simultaneous operator learning, while sensitivity analysis confirms optimal performance in medium time window scenarios.

Moreover, our proposed framework exhibits considerable potential in the realm of cross-domain emergency planning and scheduling. In emergency scenarios such as natural disasters, public health crises, or other urgent events, rapid and effective decision-making is paramount. The adaptive capability of PPO-ALNS enables it to swiftly re-optimize routes and resource allocation in response to dynamically changing conditions, thereby ensuring timely delivery of critical supplies and services. By integrating real-time data and leveraging DRL, our approach can significantly enhance emergency responsiveness, reduce logistical delays, and support coordinated dispatch across multiple domains. This innovative application bridges the gap between methodological advances and practical emergency management, underscoring the method's broad applicability and strategic value in high-stakes operational contexts.

Future research directions include investigating policy transferability across problem variants, incorporating dynamic real-world constraints, and exploring advanced neural architectures for enhanced adaptive capabilities.

Appendix A State space features

This section provides detailed descriptions of the state space features used in our PPO-ALNS framework (Table 4).

Table 4 State space features for PPO-ALNS

Feature	Description
search_progress	Search progress, ranging from 0 to 1, indicating the relative progress towards the maximum number of iterations
solution_delta	Improvement coefficient of the current iteration, ranging from -1 to 1, representing the current solution's quality
init_cost	The objective function value at the start, ranging from 0 to 1, representing the initial solution's cost
best_cost	The objective function value of the best solution found, ranging from 0 to 1, indicating the cost of the optimal solution
destroy_usage	The usage frequency of destroy operators, ranging from 0 to 1, indicating the frequency of each destroy operator's application
repair_usage	The usage frequency of repair operators, ranging from 0 to 1, indicating the frequency of each repair operator's application
demand	The demand data in the VRPTW
time_windows	The time window data in the VRPTW problem, representing the available service time windows for each customer
service_times	The service time data in the VRPTW problem, indicating the time taken to serve each customer
travel_times	The travel time data in the VRPTW problem, representing the time required to travel between customers

Appendix B Operator details

This section provides detailed descriptions of the destroy and repair operators used in our PPO-ALNS framework.

Appendix B.1 Destroy operators

- **Random Destroy**: Eliminates a randomly selected subset of customer nodes from the current solution.
- **String Destroy**: Removes continuous route segments containing randomly selected anchor customers.
- **Route Destroy**: Selects complete routes for elimination using inverse-length probability weighting, where shorter routes receive higher removal likelihood. This mechanism promotes diverse exploration of routing configurations.
- **Worst-Cost Destroy**: Prioritizes high-impact node removal through cost-benefit analysis. Incorporates a randomization factor to balance greedy elimination and stochastic exploration (Liu et al. 2024).
- **Sequence Destroy**: Disrupts route continuity by eliminating random contiguous subsequences from concatenated route representations (Li et al. 2016).

Appendix B.2 Repair operators

- **Greedy Repair**: Inserts unassigned customers into the best feasible route position that minimizes travel cost increase. If no feasible insertion exists, creates a new route for the customer.
- **Criticality-Based Repair**: Prioritizes customers by a composite criticality metric (demand magnitude, time window tightness, and depot proximity), inserting high-priority customers first to enhance solution quality.
- **Regret-k Repair**: Employs a dynamic regret mechanism (with adaptive k based on problem scale) to select insertions that maximize the cost difference between the best and subsequent insertion options, balancing immediate and future insertion costs (Qu and Bard 2012).

Appendix C Extended experimental analysis

This appendix presents additional experimental evaluations that demonstrate the adaptability, scalability, and robustness of our PPO-ALNS framework across diverse problem configurations and scales.

Appendix C.1 Generalization across data distribution types

To evaluate the generalization capability of our approach across different spatial distributions, we conducted experiments on clustered (C-type) customer distributions,

Table 5 Performance comparison on clustered customer distributions (C-type)

Method	$n = 20$		$n = 50$		$n = 100$	
	Costs	Impr	Costs	Impr	Costs	Impr
ALNS(CW)	2.61	–	6.17	–	11.72	–
Ours(CW)	**2.52**	3.26%	**6.08**	1.46%	**10.89**	7.04%

Table 6 Performance comparison for large-scale instances

Method	$n = 200$			$n = 300$		
	Costs	Impr	Time (s)	Costs	Impr	Time (s)
ALNS(CW)	39.65	–	12.25	56.00	–	26.90
Ours(CW)	**37.13**	6.36%	221.85	**52.50**	6.25%	467.70

complementing the random (R-type) distributions used in the main experiments. The clustered instances were generated by partitioning customers into multiple geographic clusters with higher intra-cluster density, reflecting real-world scenarios such as urban delivery zones or industrial districts.

The results in Table 5 demonstrate that PPO-ALNS maintains consistent performance advantages across different spatial distributions. While the improvement magnitudes vary compared to random distributions, our approach consistently outperforms the baseline ALNS across all problem sizes. The performance variation reflects the inherent structural differences between distribution types: clustered configurations naturally facilitate more efficient route construction due to spatial proximity, potentially reducing the optimization margin for any algorithm. Nevertheless, the persistent improvements validate the adaptive learning capability of our framework across diverse problem structures.

Appendix C.2 Large-scale scalability assessment

To assess the scalability of our approach for large-scale industrial applications, we conducted experiments on instances with 200 and 300 customers. The training process required approximately 8 h for 200-customer instances and 20 h for 300-customer instances on our experimental platform, demonstrating the computational investment necessary for learning effective policies at this scale.

The large-scale results in Table 6 reveal several significant findings. First, our approach maintains substantial solution quality improvements (6.36% for $n = 200$ and 6.25% for $n = 300$) even at these increased scales, demonstrating robust scalability of the learning-based operator selection mechanism. The consistency of improvement percentages across different large-scale instances validates the effectiveness of our framework for industrial applications. Second, the computational time comparison shows that our method requires more computational resources to achieve superior solution quality. This trade-off reflects the inherent complexity of learning-based optimization: while the learned policies enable more sophisticated search behavior that leads to better solutions, the computational overhead of policy evaluation and neural network inference increases with problem scale. Nevertheless, the consistent solution quality improvements across different scales validate the

effectiveness of our learning-based approach, particularly in scenarios where solution quality is prioritized over computational speed. This characteristic positions PPO-ALNS as a valuable tool for applications where solution quality is paramount, such as strategic planning or offline optimization tasks.

Author Contributions Zhaohui Wang: Conceptualization, methodology, software implementation, formal analysis, and writing-original draft. Qiao Cui: Methodology, validation, data curation, and visualization of results. Bin Tan: Software development, algorithm implementation, and experimental validation. Xiao Yang: Formal analysis, data interpretation, and manuscript reviewing and editing. Weibang Zhou: Supervision, resources, and critical review of research outcomes. Xiangsheng Huang: Project administration, funding acquisition, and supervision (corresponding author).

Funding The authors declare that no funds, grants, or other support were received during the preparation of this manuscript.

Data Availability The datasets for this study were generated using the methodology and code described in the manuscript. All code and scripts necessary to reproduce the data and experiments are publicly available on GitHub at the following repository: https://github.com/Kikujjy/ppo-alns.

Declarations

Conflict of interest The authors have no relevant financial or non-financial interests to disclose.

References

Akbarpour N, Salehi-Amiri A, Hajiaghaei-Keshteli M, Oliva D (2021) An innovative waste management system in a smart city under stochastic optimization using vehicle routing problem. Soft Comput 25(8):6707–6727. https://doi.org/10.1007/s00500-021-05669-6

Archetti C, Bouchard M, Desaulniers G (2011) Enhanced branch and price and cut for vehicle routing with split deliveries and time windows. Transp Sci 45(3):285–298. https://doi.org/10.1287/trsc.1100.0363

Baldacci R, Mingozzi A, Roberti R (2012) Recent exact algorithms for solving the vehicle routing problem under capacity and time window constraints. Eur J Oper Res 218(1):1–6. https://doi.org/10.1016/j.ejor.2011.07.037

Bogyrbayeva A, Meraliyev M, Mustakhov T, Dauletbayev B (2024) Machine learning to solve vehicle routing problems: a survey. IEEE Trans Intell Transp Syst 25(6):4754–4772. https://doi.org/10.1109/TITS.2023.3334976

Cappart Q, Moisan T, Rousseau LM, Prémont-Schwarz I, Cire AA (2021) Combining reinforcement learning and constraint programming for combinatorial optimization. In: Proceedings of the AAAI Conference on Artificial Intelligence 35(5):3677–3687. https://doi.org/10.1609/aaai.v35i5.16484

Chen B, Qu R, Bai R, Laesanklang W (2020) A variable neighborhood search algorithm with reinforcement learning for a real-life periodic vehicle routing problem with time windows and open routes. RAIRO - Oper Res 54(5):1467–1494. https://doi.org/10.1051/ro/2019080

Clarke G, Wright JW (1964) Scheduling of vehicles from a central depot to a number of delivery points. Oper Res 12(4):568–581. https://doi.org/10.1287/opre.12.4.568

Costa L, Contardo C, Desaulniers G (2019) Exact branch-price-and-cut algorithms for vehicle routing. Transp Sci 53(4):946–985. https://doi.org/10.1287/trsc.2018.0878

Da Costa P, Rhuggenaath J, Zhang Y, Akcay A, Kaymak U (2021) Learning 2-opt heuristics for routing problems via deep reinforcement learning. SN Comput Sci 2(5):388. https://doi.org/10.1007/s42979-021-00779-2

Deng J, Wang J, Wang X, Cai Y, Liu P (2024) Multi-Task Multi-Objective Evolutionary Search Based on Deep Reinforcement Learning for Multi-Objective Vehicle Routing Problems with Time Windows. SYMMETRY-BASEL 16(8):1030. https://doi.org/10.3390/sym16081030

Dixit A, Mishra A, Shukla A (2019) Vehicle Routing Problem with Time Windows Using Meta-Heuristic Algorithms: A Survey. In: Yadav N, Yadav A, Bansal JC, Deep K, Kim JH (eds) Harmony Search and Nature Inspired Optimization Algorithms, vol 741, Springer Singapore, pp 539–546, https://doi.org/10.1007/978-981-13-0761-4_52

Dorigo M, Birattari M, Stutzle T (2006) Ant colony optimization. IEEE Comput Intell Mag 1(4):28–39. https://doi.org/10.1109/MCI.2006.329691

Enthoven DL, Jargalsaikhan B, Roodbergen KJ, Uit Het Broek MAJ, Schrotenboer AH (2020) The two-echelon vehicle routing problem with covering options: city logistics with cargo bikes and parcel lockers. Comput Oper Res 118:104919. https://doi.org/10.1016/j.cor.2020.104919

Falkner JK, Schmidt-Thieme L (2020) Learning to Solve Vehicle Routing Problems with Time Windows through Joint Attention. arXiv:2006.09100

Fernandes FC, De Souza SR, Silva MAL, Borges HE, Ribeiro FF (2009) A multiagent architecture for solving combinatorial optimization problems through metaheuristics. In: 2009 IEEE International Conference on Systems, Man and Cybernetics, IEEE, pp 3071–3076, https://doi.org/10.1109/ICSMC.2009.5345934

Gambardella LM, Taillard E, Agazzi G (1999) MACS-VRPTW: A Multiple Ant Colony System for Vehicle Routing Problems with Time Windows. Technical Report, Istituto Dalle Molle Di Studi Sull Intelligenza Artificiale

Gao L, Chen M, Chen Q, Luo G, Zhu N, Liu Z (2020) Learn to Design the Heuristics for Vehicle Routing Problem. arXiv:2002.08539

Gupta A, Ghosh S, Dhara A (2022) Deep Reinforcement Learning Algorithm for Fast Solutions to Vehicle Routing Problem with Time-Windows. In: Proceedings of the 5th Joint International Conference on Data Science & Management of Data (9th ACM IKDD CODS and 27th COMAD), ACM, pp 236–240, https://doi.org/10.1145/3493700.3493723

Ho S, Haugland D (2004) A tabu search heuristic for the vehicle routing problem with time windows and split deliveries. Comput Oper Res 31(12):1947–1964. https://doi.org/10.1016/S0305-0548(03)00155-2

Hottung A, Tierney K (2020) Neural Large Neighborhood Search for the Capacitated Vehicle Routing Problem. https://doi.org/10.3233/FAIA200124, arXiv:1911.09539

Johnn SN, Darvariu VA, Handl J, Kalcsics J (2024) A graph reinforcement learning framework for neural adaptive large neighbourhood search. Comput Oper Res 172:106791. https://doi.org/10.1016/j.cor.2024.106791

Kallestad J, Hasibi R, Hemmati A, Sörensen K (2023) A general deep reinforcement learning hyperheuristic framework for solving combinatorial optimization problems. Eur J Oper Res 309(1):446–468. https://doi.org/10.1016/j.ejor.2023.01.017

Kim G, Ong YS, Heng CK, Tan PS, Zhang NA (2015) City vehicle routing problem (city VRP): a review. IEEE Trans Intell Transp Syst 16(4):1654–1666. https://doi.org/10.1109/TITS.2015.2395536

Kool W, van Hoof H, Welling M (2019) Attention, Learn to Solve Routing Problems! arXiv:1803.08475

Kosolsombat S, Ratanavilisagul C (2023) Applied Deep Reinforcement Learning for Solving the Vehicle Routing Problem with Time Windows. In: 2023 8th International Conference on Computational Intelligence and Applications (ICCIA), IEEE, pp 21–25, https://doi.org/10.1109/ICCIA59741.2023.00012

Li B, Krushinsky D, Van Woensel T, Reijers HA (2016) An adaptive large neighborhood search heuristic for the share-a-ride problem. Comput Oper Res 66:170–180. https://doi.org/10.1016/j.cor.2015.08.008

Li K, Zhang T, Wang R (2021) Deep reinforcement learning for multiobjective optimization. IEEE Trans Cybernet 51(6):3103–3114. https://doi.org/10.1109/TCYB.2020.2977661

Liu S, Sun J, Duan X, Liu G (2024) Parallel adaptive large neighborhood search based on spark to solve VRPTW. Sci Rep 14(1):23809. https://doi.org/10.1038/s41598-024-74432-2

Liu X, Chen YL, Por LY, Ku CS (2023) A systematic literature review of vehicle routing problems with time windows. Sustainability 15(15):12004. https://doi.org/10.3390/su151512004

Lopes Silva MA, De Souza SR, Freitas Souza MJ, Bazzan ALC (2019) A reinforcement learning-based multi-agent framework applied for solving routing and scheduling problems. Expert Syst Appl 131:148–171. https://doi.org/10.1016/j.eswa.2019.04.056

Martins LDC, De La Torre R, Corlu CG, Juan AA, Masmoudi MA (2021) Optimizing ride-sharing operations in smart sustainable cities: challenges and the need for agile algorithms. Comput Ind Eng 153:107080. https://doi.org/10.1016/j.cie.2020.107080

Mei H, Jingshuai Y, Teng M, Xiuli L, Ting W (2017) The modeling of milk-run vehicle routing problem based on improved c-w algorithm that joined time window. Transportation Research Procedia 25:716–728. https://doi.org/10.1016/j.trpro.2017.05.453

Nazari M, Oroojlooy A, Snyder L, Takac M (2018) Reinforcement Learning for Solving the Vehicle Routing Problem. In: Advances in Neural Information Processing Systems, Curran Associates, Inc., vol 31

Pisinger D, Ropke S (2007) A general heuristic for vehicle routing problems. Comput Oper Res 34(8):2403–2435. https://doi.org/10.1016/j.cor.2005.09.012

Qin W, Zhuang Z, Huang Z, Huang H (2021) A novel reinforcement learning-based hyper-heuristic for heterogeneous vehicle routing problem. Comput Ind Eng 156:107252. https://doi.org/10.1016/j.cie.2021.107252

Qu Y, Bard JF (2012) A grasp with adaptive large neighborhood search for pickup and delivery problems with transshipment. Comput Oper Res 39(10):2439–2456. https://doi.org/10.1016/j.cor.2011.11.016

Reijnen R, Zhang Y, Lau HC, Bukhsh Z (2024) Online control of adaptive large neighborhood search using deep reinforcement learning. In: Proceedings of the International Conference on Automated Planning and Scheduling 34:475–483. https://doi.org/10.1609/icaps.v34i1.31507

Republic DoTaLIoTaBiČBC, Jeřábek K, Majercak P, Kliestik T, Valaskova K (2016) Application of Clark and Wright's Savings Algorithm Model to Solve Routing Problem in Supply Logistics. Naše more 63(3):115–119, https://doi.org/10.17818/NM/2016/SI7

Ropke S, Pisinger D (2006) An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. Transp Sci 40(4):455–472. https://doi.org/10.1287/trsc.1050.0135

Santini A, Ropke S, Hvattum LM (2018) A comparison of acceptance criteria for the adaptive large neighbourhood search metaheuristic. J Heuristics 24(5):783–815. https://doi.org/10.1007/s10732-018-9377-x

Schulman J, Wolski F, Dhariwal P, Radford A, Klimov O (2017) Proximal Policy Optimization Algorithms. https://doi.org/10.48550/arXiv.1707.06347, arXiv:1707.06347

Shaw P (1998) Using Constraint Programming and Local Search Methods to Solve Vehicle Routing Problems. In: Goos G, Hartmanis J, Leeuwen JV, Maher M, Puget JF (eds) Principles and Practice of Constraint Programming — CP98, vol 1520, Springer Berlin Heidelberg, pp 417–431, https://doi.org/10.1007/3-540-49481-2_30

Solomon MM (1987) Algorithms for the Vehicle Routing and Scheduling Problems with Time Window Constraints. Oper Res 35(2):254–265 arXiv:170697

Toth P, Vigo D (2002) The vehicle routing problem. Society for Industrial and Applied Mathematics, https://doi.org/10.1137/1.9780898718515

Turkeš R, Sörensen K, Hvattum LM (2021) Meta-analysis of metaheuristics: quantifying the effect of adaptiveness in adaptive large neighborhood search. Eur J Oper Res 292(2):423–442. https://doi.org/10.1016/j.ejor.2020.10.045

Ursani Z, Essam D, Cornforth D, Stocker R (2011) Localized genetic algorithm for vehicle routing problem with time windows. Appl Soft Comput 11(8):5375–5390. https://doi.org/10.1016/j.asoc.2011.05.021

Voigt S (2025) A review and ranking of operators in adaptive large neighborhood search for vehicle routing problems. Eur J Oper Res 322(2):357–375. https://doi.org/10.1016/j.ejor.2024.05.033

Wang C, Cao M, Jiang H, Xiang X, Zhang X (2025) A deep reinforcement learning-based adaptive large neighborhood search for capacitated electric vehicle routing problems. IEEE Trans Emerg Top Comput Intell 9(1):131–144. https://doi.org/10.1109/TETCI.2024.3444698

Wang Z, Yan Q, Wang L, Hei X (2023) Solving the Vehicle Routing Problem with Time Window based on Reinforcement Learning. In: 2023 19th International Conference on Natural Computation, Fuzzy Systems and Knowledge Discovery (ICNC-FSKD), IEEE, pp 1–7, https://doi.org/10.1109/ICNC-FSKD59587.2023.10280869

Windras Mara ST, Norcahyo R, Jodiawan P, Lusiantoro L, Rifai AP (2022) A survey of adaptive large neighborhood search algorithms and applications. Comput Oper Res 146:105903. https://doi.org/10.1016/j.cor.2022.105903

Wu R, Wang R, Hao J, Wu Q, Wang P, Niyato D (2024) Multiobjective Vehicle Routing Optimization with Time Windows: A Hybrid Approach Using Deep Reinforcement Learning and NSGA-II. arXiv:2407.13113

Wu Y, Song W, Cao Z, Zhang J, Lim A (2022) Learning improvement heuristics for solving routing problems. IEEE Trans Neural Netw Learn Syst 33(9):5057–5069. https://doi.org/10.1109/TNNLS.2021.3068828

Zhang Y, Wang J, Zhang Z (2022) Edge-based Formulation with Graph Attention Network for Practical Vehicle Routing Problem with Time Windows. In: 2022 International Joint Conference on Neural Networks (IJCNN), IEEE, pp 01–08, https://doi.org/10.1109/IJCNN55064.2022.9892537

Zhao J, Mao M, Zhao X, Zou J (2021) A hybrid of deep reinforcement learning and local search for the vehicle routing problems. IEEE Trans Intell Transp Syst 22(11):7208–7218. https://doi.org/10.1109/TITS.2020.3003163

Zhong Y, Pan X (2007) A Hybrid Optimization Solution to VRPTW Based on Simulated Annealing. In: 2007 IEEE International Conference on Automation and Logistics, IEEE, pp 3113–3117, https://doi.org/10.1109/ICAL.2007.4339117

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Zhaohui Wang[1] · Qiao Cui[1] · Bin Tan[1] · Xiao Yang[2] · Weibang Zhou[2] · Xiangsheng Huang[3]

✉ Xiangsheng Huang
huangxiangsheng@xii.ac.cn

Qiao Cui
cuiq@chinasatnet.com.cn

1 China Satellite Network Digital Technology Co., Ltd., Xiong'an New Area, China

2 School of Integrated Circuits, University of Chinese Academy of Sciences, Beijing, China

3 Hebei Key Laboratory of Cognitive Intelligence, Xiong'an Institute of Innovation, Xiong'an New Area 071700, China